Computation steps for the sample-path dependent AIS bound for the inventory control example in [1]

March 22, 2026

1 Problem Setup

We have two discounted-cost MDPs sharing the same state space $\mathcal{S} = \{-S_{\max}, \ldots, S_{\max}\}$ and action space $\mathcal{A} = \{0, \ldots, S_{\max}\}$:

Parameter	True model M	Approximate model $\hat{M}$
Discount factor γ	0.75	0.75
Demand $D \sim \mathrm{Bin}(n, q)$	$n = 10; q = 0.4$	$n = 10; q = 0.5$
Holding cost c_h	4	3.8
Shortage cost c_s	2	2
Ordering cost p	5	5
$S_{\max}$	500	500

The per-step cost in each model is

$$c(s, a) = p \cdot a + h(s), \qquad h(s) = \begin{cases} c_h\, s & s \geq 0, \\ -c_s\, s & s < 0. \end{cases}$$

The transition kernel is deterministic given the demand realisation D:

$$s' = \mathrm{clip}(s + a - D; -S_{\max}; S_{\max}).$$

We denote by $\pi^\star$ and $\hat{\pi}^\star$ the optimal policies of M and $\hat{M}$ (both are base-stock policies with levels $\sigma^\star = 2$ and $\hat{\sigma}^\star = 3$), and by $V^\star$, $\hat{V}^\star$, $V^{\hat{\pi}^\star}$ the corresponding value functions. Our goal is a *state-dependent* bound $\alpha\colon \mathcal{S} \to \mathbb{R}_{\geq 0}$ satisfying

$$2\ alpha(s) \ \geq\ V^{\hat{\pi}^\star}(s) - V^\star(s) \qquad \forall\, s \in \mathcal{S}. \tag{1}$$

2 Inputs Required

Before the sample-path bound computation we need the following, all obtained via value iteration on the two models:

1. $\hat{V}^\star(s)$ for all s (optimal value function of $\hat{M}$),
2. $\pi^\star(s)$ for all s (optimal policy of M),
3. $\hat{\pi}^\star(s)$ for all s (optimal policy of $\hat{M}$),
4. $W_M[w] = \Pr_M(D = w)$ and $W_{\hat{M}}[w] = \Pr_{\hat{M}}(D = w)$ for $w = 0, \ldots, n$ (demand PMFs).

3 Step 1: Cost Mismatch $\varepsilon(s)$

The cost mismatch at state s is

$$\varepsilon(s) \;=\; \big|c_M(s,a) - c_{\hat{M}}(s,a)\big| \;=\; \big|h_M(s) - h_{\hat{M}}(s)\big| \tag{2}$$

where the second equality uses the fact that the ordering cost p is identical in both models, so $p\,a$ cancels. Note that ε depends only on s, not on the action a.

Substituting the holding/shortage costs:

$$\varepsilon(s) = \begin{cases} |c_h - \hat{c}_h| \cdot s = |4 - 3.8| \cdot s = 0.2\,s & \text{if } s \geq 0, \\ |c_s - \hat{c}_s| \cdot |s| = |2 - 2| \cdot |s| = 0 & \text{if } s < 0. \end{cases} \tag{3}$$

Implementation. `bounds.py`, lines 94–97:

```
h_M  = M.h_vec(states)       # h_M(s) for all s
h_Mh = M_hat.h_vec(states)   # h_Mhat(s) for all s
epsilon = np.abs(h_M - h_Mh) # shape (num_states,)
```

Output: a vector $\varepsilon \in \mathbb{R}^{|\mathcal{S}|}$ indexed by state.

4 Step 2: Transition Mismatch $\Delta(s,a)$

The transition mismatch measures how differently the two models propagate $\hat{V}^\star$:

$$\Delta(s,a) \;=\; \Big|\sum_{s'\in\mathcal{S}} \hat{V}^\star(s')\,P_M(s' \mid s,a) \;-\; \sum_{s'\in\mathcal{S}} \hat{V}^\star(s')\,P_{\hat{M}}(s' \mid s,a)\Big|. \tag{4}$$

5 Step 3: Fixed-Point Iteration for α

We now compute $\alpha(s)$ via fixed-point iteration. We first present the direct "Q-function style" formulation that matches the paper's notation exactly, then show how the H-array trick makes it computationally tractable.

5.1 Direct formulation (Q-function version)

This version uses the full transition kernel $P(s' \mid s,a)$ and follows the MDP corollary of Theorem 2 directly.

Precomputation (done once).

- **Cost mismatch** (a vector over states):

$$\varepsilon(s) = |c_M(s,a) - c_{\hat{M}}(s,a)| = |h_M(s) - h_{\hat{M}}(s)| \qquad \forall\, s \in \mathcal{S}. \tag{5}$$

- **Transition mismatch** (a matrix over state–action pairs):

$$\Delta(s,a) = \Big|\sum_{s'\in\mathcal{S}} \hat{V}^\star(s')\,P_M(s' \mid s,a) - \sum_{s'\in\mathcal{S}} \hat{V}^\star(s')\,P_{\hat{M}}(s' \mid s,a)\Big| \qquad \forall\,(s,a). \tag{6}$$

  In matrix notation, with P and $\hat{P}$ denoting the $|\mathcal{S}| \times |\mathcal{A}| \times |\mathcal{S}|$ transition arrays:

$$\boldsymbol{\Delta} = \big|P\hat{\mathbf{V}}^\star - \hat{P}\hat{\mathbf{V}}^\star\big|, \qquad \text{shape: } |\mathcal{S}| \times |\mathcal{A}|.$$

Iteration (until convergence). Initialise $\alpha^0(s) = 0$ for all s. At each iteration k:

1. **Propagate α^k through the true dynamics.** For every state–action pair (s, a), compute the expected future error under the **true** model:

$$\big[P_M \boldsymbol{\alpha}^k\big](s,a) \;=\; \sum_{s' \in \mathcal{S}} \alpha^k(s')\, P_M(s' \mid s,a). \tag{7}$$

   This is a matrix–vector product: P_M has shape $|\mathcal{S}| \times |\mathcal{A}| \times |\mathcal{S}|$ and $\boldsymbol{\alpha}^k$ has shape $|\mathcal{S}|$, giving a result of shape $|\mathcal{S}| \times |\mathcal{A}|$.

2. **Compute the error certificate β^k.** Assemble the three terms into a $|\mathcal{S}| \times |\mathcal{A}|$ matrix:

$$\boxed{\beta^k(s,a) \;=\; \underbrace{\varepsilon(s)}_{\text{cost mismatch}} \;+\; \underbrace{\gamma \sum_{s'} \alpha^k(s')\, P_M(s' \mid s,a)}_{\text{propagated future error}} \;+\; \underbrace{\Delta(s,a)}_{\text{transition mismatch}}} \tag{8}$$

   In matrix notation:

$$\boldsymbol{\beta}^k = \boldsymbol{\varepsilon} \otimes \mathbf{1}_{|\mathcal{A}|} + \gamma\, P_M \boldsymbol{\alpha}^k + \boldsymbol{\Delta}, \qquad \text{shape: } |\mathcal{S}| \times |\mathcal{A}|.$$

3. **Update α.**

   - $\alpha_{\max}$ (tighter): evaluate β^k only at the two relevant actions:

$$\alpha^{k+1}_{\max}(s) = \max\Big\{\beta^k\big(s, \pi^\star(s)\big);\; \beta^k\big(s, \hat{\pi}^\star(s)\big)\Big\}. \tag{9}$$

     In code: indexing into the two columns of $\boldsymbol{\beta}^k$ at $\pi^\star(s)$ and $\hat{\pi}^\star(s)$, then taking the element-wise max.

   - $\alpha_{\sup}$ (looser): maximise over *all* actions:

$$\alpha^{k+1}_{\sup}(s) = \max_{a \in \mathcal{A}} \beta^k(s,a). \tag{10}$$

     In code: $\boldsymbol{\alpha}^{k+1}_{\sup} = \max_{\text{axis}=1} \boldsymbol{\beta}^k$.

4. **Check convergence**: stop if $\max_s |\alpha^{k+1}(s) - \alpha^k(s)| < \texttt{tol}$.

Pseudocode (Q-function version).

```
# Precompute (once)
epsilon[s]   = |h_M(s) - h_Mhat(s)|         for all s       # |S| vector
Delta[s, a]  = |P[s,a,:] @ V_hat_star        for all (s,a)   # |S| x |A|
               - P_hat[s,a,:] @ V_hat_star|

# Iterate
alpha = zeros(|S|)
for k = 0, 1, 2, ...:
    propagated[s,a] = P[s,a,:] @ alpha      for all (s,a)   # |S| x |A|
    beta[s,a] = epsilon[s] + gamma * propagated[s,a] + Delta[s,a]

    alpha_new[s] = max( beta[s, pi_star[s]], beta[s, pi_hat_star[s]] )
    if max|alpha_new - alpha| < tol: break
    alpha = alpha_new
```

Computational cost. The bottleneck is the propagation step (7): multiplying the $|\mathcal{S}| \times |\mathcal{A}| \times |\mathcal{S}|$ transition array by the $|\mathcal{S}|$-vector $\boldsymbol{\alpha}^k$. With $|\mathcal{S}| = 1001$ and $|\mathcal{A}| = 501$:

$$\text{storage: } 1001 \times 501 \times 1001 \approx 5 \times 10^8 \text{ entries} \approx 4\ textGB(float64),$$

$$\text{per-iteration cost: } O(|\mathcal{S}|^2 \cdot |\mathcal{A}|) \approx 5 \times 10^8 \text{ multiplications.}$$

This is why we use the H-array that is introduced in the next section.

6 Step 4: The H-Array Trick (Efficient Implementation)

The Q-function version (Section 5.1) is conceptually clean but computationally infeasible at $S_{\max} = 500$. We now show how the inventory MDP's structure reduces every transition-matrix operation to a one-dimensional convolution, eliminating the need to store or multiply P.

6.1 Key observation: post-order inventory

In the inventory MDP, the next state is $s' = \text{clip}(s + a - D)$ where $D \sim \text{Bin}(n, q)$ has support $\{0, \ldots, n\}$. For any function f over states:

$$\sum_{s'} f(s')\, P(s' \mid s, a) = \sum_{w=0}^{n} W[w] \cdot f\big(\text{clip}(s + a - w)\big). \tag{11}$$

The right-hand side depends on s and a **only through the sum** $z = s + a$, which is the *post-order inventory level* (inventory after ordering, before demand). We precompute the **H-array**:

$$H_f[z] = \sum_{w=0}^{n} W[w] \cdot f\big(\text{clip}(z - w)\big), \qquad z \in \{-S_{\max}, \ldots, 2S_{\max}\}, \tag{12}$$

so that $\sum_{s'} f(s')P(s' \mid s, a) = H_f[s + a]$.

Remark 1. *The index z ranges over $\{-S_{\max}, \ldots, 2S_{\max}\}$ because $s + a \in \{-S_{\max} + 0, \ldots, S_{\max} + S_{\max}\}$. The H-array has length $3S_{\max} + 1 = 1501$, compared to the $1001 \times 501 = 501{,}501$ entries of the full $[P\mathbf{f}](s, a)$ matrix.*

6.2 Mapping each Q-function step to its H-array equivalent

We now show how each step from Section 5.1 translates.

Transition mismatch. The Q-function version computes $\Delta(s, a) = |[P_M \hat{\mathbf{V}}^\star](s, a) - [P_{\hat{M}} \hat{\mathbf{V}}^\star](s, a)|$. Using H-arrays:

$$H_M[z] = \sum_{w=0}^{n} W_M[w] \cdot \hat{V}^\star\big(\text{clip}(z - w)\big), \tag{13}$$

$$H_{\hat{M}}[z] = \sum_{w=0}^{n} W_{\hat{M}}[w] \cdot \hat{V}^\star\big(\text{clip}(z - w)\big), \tag{14}$$

and

$$H_\Delta[z] = |H_M[z] - H_{\hat{M}}[z]|, \qquad \Delta(s, a) = H_\Delta[s + a]. \tag{15}$$

Instead of an $|\mathcal{S}| \times |\mathcal{A}|$ matrix, we store a single vector of length $3S_{\max} + 1$.

Implementation. `bounds.py`, lines 100–107:

```
H_M  = np.zeros(H_len)   # H_len = 3*s_max + 1
H_Mh = np.zeros(H_len)
for z in range(-s_max, 2*s_max + 1):
    for w in range(n_demand):
        ns = min(max(z - w, -s_max), s_max)
        H_M[z + s_max]  += W_M[w]  * V_hat_star[ns + s_max]
        H_Mh[z + s_max] += W_Mh[w] * V_hat_star[ns + s_max]
H_delta = np.abs(H_M - H_Mh)
```

Propagation of α. The Q-function version computes $[P_M \boldsymbol{\alpha}^k](s,a) = \sum_{s'} \alpha^k(s') P_M(s'|s,a)$. Using the H-array:

$$H_\alpha[z] = \sum_{w=0}^{n} W_M[w] \cdot \alpha^k\big(\mathrm{clip}(z-w)\big), \qquad \sum_{s'} \alpha^k(s') P_M(s'|s,a) = H_\alpha[s+a]. \tag{16}$$

One vector of length $3S_{\max}+1$ replaces the full $|\mathcal{S}| \times |\mathcal{A}|$ matrix–vector product.

Implementation. `bounds.py`, lines 115–119:

```
H_alpha = np.zeros(H_len)
for z in range(-s_max, 2*s_max + 1):
    for w in range(n_demand):
        ns = min(max(z - w, -s_max), s_max)
        H_alpha[z + s_max] += W_M[w] * alpha[ns + s_max]
```

Remark 2 (Why the true model?)**.** *The propagation uses W_M (not $W_{\hat{M}}$) because $\alpha(s)$ bounds the suboptimality gap under the* true *model's dynamics. The approximate model's influence is already captured in H_Δ.*

Error certificate β. The Q-function formula (8) becomes:

$$\boxed{\beta(s,a) \;=\; \underbrace{\varepsilon(s)}_{\text{cost mismatch}} \;+\; \underbrace{\gamma \cdot H_\alpha[s+a]}_{\text{propagated future error}} \;+\; \underbrace{H_\Delta[s+a]}_{\text{transition mismatch}}} \tag{17}$$

This is identical to (8), but with $H_\alpha[s+a]$ replacing $\sum_{s'} \alpha^k(s') P_M(s'|s,a)$ and $H_\Delta[s+a]$ replacing $\Delta(s,a)$ — both are just look-ups into precomputed arrays.

Update of α.

Variant 1: $\alpha_{\max}$ (tighter). Maximise only over the two relevant policies:

$$\alpha_{\max}^{k+1}(s) \;=\; \max\Big\{\beta\big(s,\pi^\star(s)\big);\; \beta\big(s,\hat{\pi}^\star(s)\big)\Big\}. \tag{18}$$

This suffices for the bound (1) because the suboptimality gap $V^{\hat{\pi}^\star}(s) - V^\star(s)$ involves only these two policies.

Implementation. `bounds.py`, lines 122–131:

```
alpha_new_max = np.zeros(num_states)
for i, s in enumerate(states):
    s_int = int(s)
    best = -inf
    for a in [pi_star[i], pi_hat_star[i]]:
        z_idx = min(s_int + a, 2*s_max) + s_max
        beta_sa = epsilon[i] + gamma * H_alpha[z_idx] + H_delta[z_idx]
        if beta_sa > best:
            best = beta_sa
    alpha_new_max[i] = best
```

Variant 2: α_{sup} (looser). Maximize over *all* actions:

$$\alpha_{\text{sup}}^{k+1}(s) \;=\; \max_{a\in\mathcal{A}} \beta(s,a) \;=\; \varepsilon(s) \;+\; \max_{a=0,\ldots,S_{\max}} \big[\gamma \cdot H_\alpha[s+a] + H_\Delta[s+a]\big]. \tag{19}$$

The second equality uses the fact that $\varepsilon(s)$ does not depend on a. For a given state s, the maximization ranges over $z = s + a$ for $a = 0, \ldots, S_{\max}$, i.e., $z \in \{s, \ldots, s + S_{\max}\}$.

Implementation. `bounds.py`, lines 137–144:

```
combo = gamma * H_alpha + H_delta    # shape (H_len,)
alpha_new_sup = np.zeros(num_states)
for i, s in enumerate(states):
    s_int = int(s)
    z_lo = s_int + s_max             # array index for a=0
    z_hi = min(s_int + s_max, 2*s_max) + s_max  # for a=s_max
    alpha_new_sup[i] = epsilon[i] + np.max(combo[z_lo : z_hi + 1])
```

By construction, $\alpha_{\max}(s) \le \alpha_{\text{sup}}(s)$ for all s, since $\alpha_{\max}$ maximises over a *subset* of the actions.

Convergence check. After computing α^{k+1}, check:

$$\delta^k \;=\; \max_{s\in\mathcal{S}} \big|\alpha^{k+1}(s) - \alpha^k(s)\big| \;<\; \texttt{tol} = 10^{-10}. \tag{20}$$

If satisfied, terminate. Otherwise, set $\alpha^k \leftarrow \alpha^{k+1}$ and return to the propagation step.

Implementation. `bounds.py`, lines 146–152:

```
diff = np.max(np.abs(alpha_new_max - alpha))
history.append(diff)
alpha = alpha_new_max
if diff < tol:
    break
```

6.3 Correspondence between the two formulations

The table below summarises how each quantity in the Q-function version maps to its H-array equivalent.

Quantity	**Q-function version**	**H-array version**
Transition mismatch	$\Delta(s,a) = \lvert[P_M \hat{\mathbf{V}}^\star - P_{\hat{M}} \hat{\mathbf{V}}^\star](s,a)\rvert$	$H_\Delta[s+a]$
Propagated error	$[P_M \boldsymbol{\alpha}^k](s,a)$	$H_\alpha[s+a]$
Error certificate	$\beta^k(s,a) = \varepsilon(s) + \gamma[P_M \boldsymbol{\alpha}^k](s,a) + \Delta(s,a)$	$\varepsilon(s) + \gamma H_\alpha[s+a] + H_\Delta[s+a]$
Storage	$\lvert\mathcal{S}\rvert \times \lvert\mathcal{A}\rvert$ matrices ($\sim$501K entries)	vectors of length $3S_{\max}+1$ (1501 entries)
Per-iteration cost	$O(\lvert\mathcal{S}\rvert^2 \cdot \lvert\mathcal{A}\rvert) \approx 5 \times 10^8$	$O(n \cdot (3S_{\max}+1)) \approx 1.7 \times 10^4$

The H-array trick exploits the fact that in the inventory MDP, $P(s'|s,a)$ depends on (s,a) only through the post-order level $z = s+a$. This collapses the two-dimensional (s,a) index into a single index z, reducing storage by a factor of $\sim 334\times$ and computation by a factor of $\sim 29{,}000\times$ per iteration.

7 Convergence Guarantee

Define the operator $\mathcal{T}\colon \mathbb{R}^{|\mathcal{S}|} \to \mathbb{R}^{|\mathcal{S}|}$ by

$$(\mathcal{T}\alpha)(s) \;=\; \max_{a \in A(s)} \Big[\varepsilon(s) + \gamma \sum_{s'} \alpha(s')\, P_M(s' \mid s,a) + H_\Delta[s+a]\Big],$$

where $A(s) = \{\pi^\star(s), \hat{\pi}^\star(s)\}$ for the $\alpha_{\max}$ variant and $A(s) = \mathcal{A}$ for $\alpha_{\sup}$.

For any $\alpha, \alpha' \in \mathbb{R}^{|\mathcal{S}|}$:

$$\begin{aligned}\big|(\mathcal{T}\alpha)(s) - (\mathcal{T}\alpha')(s)\big| \;&\le\; \gamma \max_{a \in A(s)} \sum_{s'} \big|\alpha(s') - \alpha'(s')\big|\, P_M(s' \mid s,a) \\ &\le\; \gamma\, \|\alpha - \alpha'\|_\infty,\end{aligned}$$

since $P_M(\cdot \mid s,a)$ is a probability distribution. Hence $\mathcal{T}$ is a γ-contraction in sup-norm, and by Banach's fixed-point theorem

$$\|\alpha^{k+1} - \alpha^\star\|_\infty \;\le\; \gamma^k\, \|\alpha^1 - \alpha^0\|_\infty.$$

With $\gamma = 0.75$, reaching tolerance 10^{-10} requires roughly $k \approx \log(10^{-10})/\log(0.75) \approx 80$ iterations (in practice: 94 iterations).

8 Full Algorithm

Algorithm 1 Sample-Path Dependent AIS Bound

Require: M, $\hat{M}$, $\hat{V}^\star$, $\pi^\star$, $\hat{\pi}^\star$, tolerance `tol`
Ensure: $\alpha_{\max}$, $\alpha_{\sup}$ such that $2\alpha(s) \geq V^{\hat{\pi}^\star}(s) - V^\star(s)$ for all s

Precompute cost mismatch: $\varepsilon(s) \leftarrow |h_M(s) - h_{\hat{M}}(s)|$ for all s *Eq.* (2)

Precompute transition mismatch H-array:
for $z = -S_{\max}$ **to** $2S_{\max}$ **do**
  $H_M[z] \leftarrow \sum_{w=0}^{n} W_M[w] \cdot \hat{V}^\star(\text{clip}(z-w))$
  $H_{\hat{M}}[z] \leftarrow \sum_{w=0}^{n} W_{\hat{M}}[w] \cdot \hat{V}^\star(\text{clip}(z-w))$
end for
$H_\Delta[z] \leftarrow |H_M[z] - H_{\hat{M}}[z]|$ for all z *Eq.* (15)

Fixed-point iteration:
$\alpha^0(s) \leftarrow 0$ for all s
for $k = 0, 1, 2, \ldots$ **do**
  Build $H_\alpha[z] \leftarrow \sum_{w=0}^{n} W_M[w] \cdot \alpha^k(\text{clip}(z-w))$ for all z *Eq.* (16)
  for each state s **do**
    $\alpha_{\max}^{k+1}(s) \leftarrow \max_{a \in \{\pi^\star(s), \hat{\pi}^\star(s)\}} \left[\varepsilon(s) + \gamma H_\alpha[s+a] + H_\Delta[s+a]\right]$ *Eq.* (18)
    $\alpha_{\sup}^{k+1}(s) \leftarrow \varepsilon(s) + \max_{a=0}^{S_{\max}} \left[\gamma H_\alpha[s+a] + H_\Delta[s+a]\right]$ *Eq.* (19)
  end for
  if $\max_s |\alpha_{\max}^{k+1}(s) - \alpha^k(s)| <$ `tol` **then**
    break
  end if
end for
return $\alpha_{\max}$, $\alpha_{\sup}$

9 Computational Complexity

Let $N = 2S_{\max} + 1 = |\mathcal{S}|$ and $n = 10$ (demand support size).

Step	Computed once	Per iteration
Cost mismatch ε	$O(N)$	—
Transition mismatch H_Δ	$O(n \cdot (3S_{\max} + 1))$	—
Propagation H-array H_α	—	$O(n \cdot (3S_{\max} + 1))$
$\alpha_{\max}$ update	—	$O(2N)$
$\alpha_{\sup}$ update	—	$O(N \cdot S_{\max})$

With $S_{\max} = 500$, $n = 10$, and ~ 94 iterations, the total cost is dominated by the $\alpha_{\sup}$ update at $O(N \cdot S_{\max}) \approx 5 \times 10^5$ per iteration. The $\alpha_{\max}$ variant requires only $O(N)$ per iteration and converges in the same number of iterations.

The key saving is that no transition matrix (which would require $O(N^2 \cdot |\mathcal{A}|) \approx 5 \times 10^8$ entries) is ever formed or stored.

10 Error Decomposition at the Fixed Point

At convergence, $\alpha = \mathcal{T}\alpha$, so for the action $a = \hat{\pi}^\star(s)$ (which achieves the max at most states in the operating region):

$$\alpha(s) \;=\; \underbrace{\varepsilon\big(s, \hat{\pi}^\star(s)\big)}_{\text{cost mismatch}} \;+\; \underbrace{\gamma \sum_{s'} \alpha(s')\, P_M\big(s' \mid s, \hat{\pi}^\star(s)\big)}_{\text{propagated future error}} \;+\; \underbrace{\Delta\big(s, \hat{\pi}^\star(s)\big)}_{\text{transition mismatch on } \hat{V}^\star} \quad . \tag{21}$$

This decomposition reveals which source of model mismatch dominates at each state:

- For $s \geq 0$: the cost mismatch $\varepsilon(s) = 0.2\, s$ grows linearly, eventually dominating.
- For $s < 0$: the cost mismatch is zero (c_s is identical), so the bound is driven entirely by the transition mismatch Δ and propagated error.
- Near the base-stock levels ($s \in \{-7, \ldots, 3\}$): the propagated future error $\gamma\mathbb{E}[\alpha(s')]$ is the dominant term, since the local cost and transition mismatches are small.

11 Numerical Results

Method	Max bound in $\{-7ldots, 3\}$	Max bound (all s)
True gap $V^{\hat{\pi}^\star} - V^\star$	2.01	2.01
$2\alpha_{\max}$ (sample-path)	54.98	902.40
$2\alpha_{\sup}$ (sample-path)	718.58	917.38
Weighted-norm ($\ell = 0.015$)	64.32	1568.18
Sup-norm ($\ell = 0$)	891.20	891.20

The operating region $\{-7, \ldots, 3\}$ is determined by the approximate policy $\hat{\pi}^\star$: it orders up to the base-stock level $\hat{\sigma}^\star = 3$, and the maximum demand realisation is $n = 10$, giving a lower bound of $3 - 10 = -7$.

- $2\alpha_{\max}$ is $\mathbf{0.85}\times$ the weighted-norm bound (tighter by 15%).
- $2\alpha_{\max}$ is $\mathbf{0.062}\times$ the sup-norm bound (tighter by a factor of 16).
- The bound is valid everywhere: $\min_s[2\alpha_{\max}(s) - (V^{\hat{\pi}^\star}(s) - V^\star(s))] = 40.27 \geq 0$.
- $\alpha_{\max}(s) \leq \alpha_{\sup}(s)$ for all s (as expected).

12 Remark: When the True Transition Kernel Is Unknown

Both the sample-path bound and the weighted-norm bound from the model approximation paper require access to the true model's one-step transition probabilities $P_M(s' \mid s, a)$. Specifically:

- The transition mismatch $\Delta(s, a)$ evaluates $\sum_{s'} \hat{V}^\star(s') P_M(s' \mid s, a)$.
- The propagation step computes $\sum_{s'} \alpha(s') P_M(s' \mid s, a)$.
- The weighted contraction coefficient κ requires $\sum_{s'} w(s') P_M(s' \mid s, a)$.

In settings where P_M is not available but one has structural knowledge about how P_M and $P_{\hat{M}}$ may differ, these quantities can be *upper bounded* without evaluating P_M directly. For the transition mismatch, a standard bound via total variation distance gives:

$$\Delta(s,a) \;=\; \Big|\sum_{s'} \hat{V}^\star(s')\big[P_M(s'|s,a) - P_{\hat{M}}(s'|s,a)\big]\Big| \;\le\; \|\hat{V}^\star\|_\infty \cdot \big\|P_M(\cdot|s,a) - P_{\hat{M}}(\cdot|s,a)\big\|_{\mathrm{TV}}, \quad (22)$$

where $\|\cdot\|_{\mathrm{TV}}$ denotes the total variation distance (half the L^1 norm). If one has an upper bound $\bar{\delta}(s,a)$ on the total variation distance — for example, from knowing that the model parameters lie in some uncertainty set — then (22) provides a computable surrogate for $\Delta(s,a)$ that does not require evaluating P_M.

Similarly, the propagation term can be routed through $P_{\hat{M}}$ with a correction:

$$\sum_{s'} \alpha(s') P_M(s'|s,a) \;\le\; \sum_{s'} \alpha(s') P_{\hat{M}}(s'|s,a) \;+\; \|\alpha\|_\infty \cdot \big\|P_M(\cdot|s,a) - P_{\hat{M}}(\cdot|s,a)\big\|_{\mathrm{TV}}.$$

The resulting bounds are looser but remain valid, and they connect to the robust/distributionally-robust MDP literature where one optimises over a worst-case transition kernel within an uncertainty set around $P_{\hat{M}}$.

References

[1] B. Bozkurt, A. Mahajan, A. Nayyar, and Y. Ouyang, "Model approximation in mdps with unbounded per-step cost," *IEEE Transactions on Automatic Control*, vol. 70, no. 7, pp. 4624–4639, 2025.